CSE 231 — Assignment 5

SimpleMultithreader | Design Document

Group Information

Group ID: 30
Person 1: Ishaan Gupta (2025603)
Person 2: Rahul Hadli (2025614)

1. Objective

The objective is to implement a header-only C++11 library that simplifies parallel loop execution using POSIX Pthreads. The library exposes one-dimensional and two-dimensional `parallel_for()` APIs that accept lambda expressions, distribute iterations across the requested number of workers, wait for all work to complete, and report execution time for each invocation.

2. Files in the Submission

- **simple-multithreader.h:** shared Pthread execution engine, both parallel_for overloads, worker entry point, timing, error checks, and demonstration wrapper.
- **vector.cpp:** supplied one-dimensional vector addition program, retained unchanged.
- **matrix.cpp:** supplied matrix initialization and two-dimensional multiplication program, retained unchanged.
- **Makefile:** supplied build rules, retained unchanged.

3. Design and Implementation

3.1 Shared execution engine

Both public APIs delegate to `smt_run_parallel()`. The helper validates the requested thread count and iteration count, divides the total iterations into balanced, non-overlapping half-open ranges, creates `numThreads - 1` POSIX threads, and executes the remaining partition on the calling/main thread. Therefore, the total number of participating execution threads is exactly the requested count, including the main thread.

3.2 Worker data and synchronization

Each SMTWorker stores the beginning and end of its assigned range and a pointer to the callable body. The Pthread entry function executes the worker's range. Worker descriptors and the callable remain alive until successfully created threads have been joined. `pthread_join()` waits for worker completion before the API returns.

3.3 One-dimensional parallel_for

The one-dimensional overload accepts the half-open interval [low, high). It checks that the upper bound is not below the lower bound, computes the iteration count using a wider integer type, and maps each worker offset back to the original loop index before invoking the supplied lambda.

3.4 Two-dimensional parallel_for

The two-dimensional overload validates both loop bounds and checks for overflow when calculating the total iteration-space size. It flattens the rectangular iteration space into rows × columns linear indices. Each linear index is converted back to row and column offsets using division and remainder, then shifted by the supplied lower bounds. This covers the two-dimensional iteration space in row-major order while allowing the shared engine to partition it evenly.

3.5 Timing

The implementation uses `std::chrono::steady_clock` to record a start time before worker setup and an end time after the work has completed and worker threads have been joined. It prints the total elapsed wall-clock time for each parallel_for invocation. An empty iteration space prints zero seconds.

3.6 Error handling

The implementation checks invalid thread counts, invalid loop bounds, and overflow in the two-dimensional iteration-space calculation. If creation of a Pthread fails, that worker's partition is executed by the calling thread so that work is not omitted. A join failure is reported and raised as a runtime error. The implementation uses POSIX Pthread APIs and does not use std::thread, std::async, or a thread pool.

4. Build and Testing

The supplied Makefile builds both sample programs on a Unix-like environment with a C++11 compiler and Pthreads. The assignment instructions specify using Linux or WSL rather than macOS for OS assignment work.

```
make
./vector 1 10000
./vector 2 10000
./matrix 1 64
./matrix 2 64
```

The sample programs were compiled using the supplied Makefile and run with one and two workers. All four test runs completed successfully and printed their respective Test Success messages. Each parallel_for invocation also printed execution time. Performance comparisons should use the same machine and input size over repeated runs because timings vary with system load and hardware.

5. Member Contributions

5.1 Ishaan Gupta (2025603) — Person 1

- Designed and implemented the shared POSIX Pthread execution mechanism used by both parallel loop APIs.
- Implemented the one-dimensional parallel_for() overload, including calculation and partitioning of the one-dimensional iteration range.
- Implemented the common worker assignment and execution flow, including Pthread creation, execution of the caller's partition, joining, and synchronization.
- Integrated the shared execution mechanism into simple-multithreader.h and reviewed its interaction with the two-dimensional overload.

5.2 Rahul Hadli (2025614) — Person 2

- Implemented the two-dimensional parallel_for() overload.
- Designed the flattening of the two-dimensional iteration space and mapping of linear indices back to row and column coordinates.
- Implemented and reviewed execution-time measurement for parallel_for invocations.
- Tested the vector and matrix sample programs with different worker counts, checked output correctness, and reviewed integration and error-handling behaviour.

6. Private GitHub Repository

Repository: https://github.com/ishaangupta-beep/OS-assignment-5
Required visibility: Private.